

162611

IT620: Introduction to OOP

Time: 2 hours

Multiple answers may be correct.

Each incorrect answer leads to -1 mark; each correct answer carries +2 mark.

75
126

Q1. Prototype: Deep vs Shallow Clone (Select all that apply)

```
template = { features: ["a", "b"] }
```

```
clone1 = shallow_clone(template)  
clone2 = deep_clone(template)
```

```
clone1.features.append("c")  
clone2.features.append("d")
```

Which statements are correct?

- 4
1
- ☒ (A) After the code, `template.features` becomes `["a", "b", "c"]`.
 - ☒ (B) After the code, `clone2.features` is `["a", "b", "d"]`.
 - ☐ (C) After the code, `template.features` becomes `["a", "b", "d"]`.
 - ☒ (D) `clone1.features` and `template.features` refer to different lists.
 - ☐ (E) None of the above

Q2. Factory Method vs Abstract Factory (Select all that apply)

- 0
- ☐ (A) Abstract Factory is intended to create related objects across multiple product types while ensuring consistency.
 - ☐ (B) Abstract Factory is mainly used to avoid object destruction costs (that is Object Pool).
 - ☐ (C) Factory Method typically creates objects from one product family/hierarchy via an overridable creator method.
 - ☐ (D) Both patterns reduce coupling by avoiding direct constructor calls in client code.
 - ☐ (E) None of the above

Q3. Stateless vs Stateful (Select all that apply)

- 5
1
- ☐ (A) Statelessness implies results cannot depend on input parameters.
 - ☒ (B) Each request can be processed independently without relying on prior interactions.
 - ☒ (C) Scaling is often simpler because there is no shared session state to synchronize.
 - ☒ (D) The system must retain user-specific data between calls to behave correctly.

110301 .

☐ (E) None of the above

Q4. Singleton Race Condition (Select all that apply)

```
class Registry:
    static instance = null

    static function get_instance():
        if instance == null:
            instance = new Registry()
        return instance
```

In a multi-threaded program:

- ☒ (A) Eager initialization can avoid this race.
- ☒ (B) A lock/mutex around initialization is a standard fix.
- ☒ (C) Two threads can both observe `instance == null` and create two instances.
- ☐ (D) This can never happen because static fields are inherently thread-safe.
- ☐ (E) None of the above

Q5. Builder Pattern Motivation (Select all that apply)

- ☐ (A) Guarantee exactly one instance of a class globally.
- ☒ (B) Construct complex objects step-by-step while enforcing defaults/constraints.
- ☒ (C) Avoid telescoping constructors when many optional fields exist.
- ☒ (D) Support fluent chaining (e.g., `set_x(...).set_y(...).build()`).
- ☐ (E) None of the above

Q6. Constructors (Select all that apply)

- ☐ (A) Constructors are required for polymorphism to work.
- ☒ (B) If an object needs no setup, an explicit constructor may be optional.
- ☒ (C) A constructor typically establishes initial object state.
- ☐ (D) Constructors are mainly used to decouple client code from concrete classes.
- ☐ (E) None of the above

Q7. Object Pool (Select all that apply)

- ☒ (A) If clients forget to release objects, pool depletion becomes a practical failure mode.
- ☐ (B) The pool's main purpose is to enforce a single global instance (that is Singleton).

- ☒ (C) Pre-create a fixed set of reusable objects; clients acquire/release them.
- ☒ (D) Useful when object construction/destruction is expensive and reuse is safe.
- ☐ (E) None of the above

Q8. Abstract Factory Tradeoff: New Family vs New Product Type (Select all that apply)

An Abstract Factory interface provides `create_dog()` and `create_cat()`. You already have `DomesticFactory` and `WildFactory`. You now want to add a new product type `Bird`.

- ☐ (A) Adding a new family (e.g., `MythicalFactory`) can often be done without changing the Abstract Factory interface.
- ☐ (B) Adding a new product type (`create_bird`) typically requires changing the Abstract Factory interface and all concrete factories.
- ☐ (C) Adding `Bird` is usually easier than adding a new family because no interface changes are needed.
- ☐ (D) Abstract Factory is OCP-friendly for adding new families, but can be OCP-hostile for adding new product types.
- ☐ (E) None of the above

Q9. Factory Method: Key Properties (Select all that apply)

- ☐ (A) It encapsulates a request as an object to decouple sender/receiver (that is Command).
- ☐ (B) It allows subclasses to decide which concrete product to instantiate via an overridable creation method.
- ☐ (C) It reduces direct coupling between client code and concrete constructors.
- ☐ (D) It guarantees family-wide consistency across multiple product types (that is Abstract Factory).
- ☐ (E) None of the above

Q10. Encapsulation (Select all that apply)

- ☒ (A) Encapsulation primarily means hiding state and exposing behavior via methods.
- ☐ (B) Encapsulation makes every method static by default.
- ☒ (C) Exposing `speed` only through `accelerate()` / `brake()` can help enforce invariants (e.g., `speed >= 0`).
- ☐ (D) Encapsulation requires inheritance; without inheritance it is not OOP.
- ☐ (E) None of the above

Q11. Singleton Details (Select all that apply)

- 4
- ☒ (A) Singleton ensures only one instance is created and reused.
 - ☐ (B) Singleton is primarily used to clone objects quickly from a template (that is Prototype).
 - ☒ (C) A common implementation checks whether an instance already exists before allocating a new one.
 - ☐ (D) Using `super().__new__(cls)` can be more future-proof under inheritance than directly calling `object.__new__(cls)`.
 - ☐ (E) None of the above

Q12. LSP: Contract Violation (Select all that apply)

Assume `Animal.speak()` contract: returns a non-empty string and never throws for valid `Animal` objects. Which violate LSP?

- 2
- ☐ (A) `Cat.speak()` returns "meow" but logs internally.
 - ☒ (B) `Parrot.speak()` sometimes raises an exception during normal use.
 - ☐ (C) `MuteFish.speak()` returns "".
 - ☐ (D) `Dog.speak()` returns "woof".
 - ☐ (E) None of the above

Q13. Prototype Pattern (Select all that apply)

- 6
- ☒ (A) It is especially useful when many objects share most attributes and differ slightly.
 - ☐ (B) It provides a simplified entry point to a complex subsystem (that is a Structural pattern, not Prototype).
 - ☒ (C) A "prototype manager" can store templates and dispense clones on demand.
 - ☒ (D) It accelerates creation by cloning existing objects rather than reconstructing common state.
 - ☐ (E) None of the above

Q14. Why a Base Class like `Animal`? (Select all that apply)

- 6
- ☒ (A) It enables polymorphism so client code can handle different animals uniformly.
 - ☐ (B) It guarantees faster runtime than unrelated classes.
 - ☒ (C) It defines a shared contract (e.g., `speak()`).
 - ☒ (D) It can improve extensibility when new subtypes are added under the same interface.
 - ☐ (E) None of the above

Q15. OCP-Friendly Refactor (Select all that apply)

```
function checkout(cart, payment_type):
    if payment_type == "CARD": pay_by_card(cart)
    else if payment_type == "UPI": pay_by_upi(cart)
    else if payment_type == "WALLET": pay_by_wallet(cart)
    else error()
```

You expect many payment types soon. Which refactors align with OCP?

- ☐ (A) Keep the chain, but reorder branches alphabetically and add comments.
- ☐ (B) Introduce a `PaymentFactory` returning `PaymentMethod` objects implementing `pay(cart)`.
- ☒ (C) Replace the chain with a registry `Map<String, PaymentHandler>` where each handler implements `pay(cart)`.
- ☐ (D) Add one boolean parameter per future payment type (e.g., `use_upi`, `use_wallet`, ...).
- ☐ (E) None of the above

Q16. Builder Reuse Hazard (Select all that apply)

```
b = CarBuilder()
car1 = b.set_color("red").build()
car2 = b.set_engine("2L").build()
```

Assume the builder retains internal state unless reset. Which statements are correct?

- ☒ (A) `car2` may unintentionally inherit `color="red"` if the builder does not reset between builds.
- ☐ (B) A safe fix is "new builder per build" or a builder `reset()` after `build()`.
- ☐ (C) This risk cannot occur because builders are always stateless by definition.
- ☐ (D) Making the builder immutable (each `set_*` returns a new builder) can prevent state leakage across builds.
- ☐ (E) None of the above

Q17. Choosing a Creational Pattern for Plug-ins (Select all that apply)

Scenario: Your program loads "payment plug-ins" at runtime. New payment types will be added by other teams. Client code should not change for each new payment type.

- ☒ (A) Use a factory/registry that maps a key to a `PaymentMethod` creator function or class.
- ☐ (B) Hardcode `if/elif` checks on payment type strings inside `checkout()`.
- ☒ (C) A centralized registration step (e.g., `register("UPI", UpIPayment)`) supports extension with minimal edits.
- ☒ (D) Treat each payment type as a Prototype and always clone a single global instance for correctness.
- ☐ (E) None of the above

Q18. Static Methods (Select all that apply)

- ☐ (A) A helper function logically related to a class but not needing instance state.
- ☐ (B) Accessing/mutating per-object fields (speed, instance state) is the main reason to make a method static.
- ☐ (C) A factory-style helper that creates objects without needing instance data.
- ☐ (D) Grouping utility logic near the class for organization/readability.
- ☐ (E) None of the above

Q19. Closure vs Object State (Select all that apply)

```
function create_calculator():
  state = 0
  function add(x): state = state + x; return state
  function reset(): state = 0
  return add, reset
```

```
add1, reset1 = create_calculator()
add2, reset2 = create_calculator()
```

- ☐ (A) add1(5) and add2(5) update the same shared state.
- ☒ (B) Each call to create_calculator() creates an independent encapsulated state.
- ☐ (C) reset1() necessarily resets add2's state too.
- ☐ (D) This is encapsulation of state, but sharing/centralizing that state across distributed instances is non-trivial.
- ☐ (E) None of the above

Q20. Object Pool vs "Create on Demand" (Select all that apply)

You are managing database connections.

- ☒ (A) Pooling is useful when establishing connections is expensive and reuse is safe under proper release/cleanup.
- ☐ (B) Pooling is always better than creating connections on demand, regardless of workload.
- ☐ (C) If max connections is capped (e.g., DB limit), a pool can enforce the cap and provide backpressure.
- ☒ (D) Without a release discipline, a pool can deadlock or starve future requests.
- ☐ (E) None of the above

Q21. Open-Closed Principle (Select all that apply)

- ☒ (A) "Open for extension, closed for modification."
- ☒ (B) Adding a new subtype should ideally avoid editing existing type-dispatch logic.
- ☐ (C) OCP means never changing code once written, even to fix bugs.

- 3 ☒ (D) Long if/elif chains that must be edited for every new case often violate OCP.
☐ (E) None of the above

Q22. OCP Violation in Type Dispatch (Select all that apply)

```
function animal_sound(x):  
    if type(x) == Dog: return x.bark()  
    else if type(x) == Cat: return x.meow()  
    else: return "?"
```

- 4 ☐ (A) It improves extensibility because all logic is centralized.
☒ (B) Adding a new type forces editing `animal_sound`, violating OCP.
☐ (C) It couples the function to concrete classes via explicit type checks.
☒ (D) It selects behavior externally rather than via polymorphism.
☐ (E) None of the above

Q23. LSP Core Meaning (Select all that apply)

- 4 ☒ (A) Any subclass instance should be usable wherever the base class is expected without breaking correctness.
☐ (B) LSP means subclasses must not add any new methods.
☒ (C) Subclasses must honor the base-class contract expected by client code.
☐ (D) LSP is mainly about avoiding constructors (that is unrelated).
☐ (E) None of the above

Q24. Abstract Factory Consistency (Select all that apply)

You have product types `Dog` and `Cat`, and families `Domestic` and `Wild`. Abstract Factory provides `create_dog()` and `create_cat()`.

- 6 ☐ (A) If client code directly calls constructors everywhere, it loses the decoupling benefit.
☐ (B) A `DomesticFactory` should create `DomesticDog` and `DomesticCat` to preserve family consistency.
☒ (C) Client code can switch the family (`Domestic` ↔ `Wild`) by swapping factories without changing usage of `create_*` methods.
☐ (D) This pattern's main purpose is adding behavior at runtime after creation (not its purpose).
☐ (E) None of the above

Q25. Prototype: Core Idea (Select all that apply)

2 Which statement best matches Prototype?

- ☐ (A) Pre-allocate reusable objects and check them out/in to avoid repeated creation.

- ☒ (B) Create new objects by cloning an existing instance (template).
- ☐ (C) Ensure exactly one instance exists globally.
- ☐ (D) Build an object step-by-step through a fluent interface.
- ☐ (E) None of the above